

Fan Director Studio — API Specification (OpenAPI 3.0)

```

openapi: 3.0.0
info:
  title: Fan Director Studio API
  version: 1.0.0
  description: Boutique creator commission platform with AI Director assistance
  contact:
    name: API Support
    email: support@fandirectorstudio.com
  license:
    name: MIT

servers:
- url: https://api.fandirectorstudio.com/v1
  description: Production
- url: http://localhost:3000/v1
  description: Development

tags:
- name: Authentication
  description: User login, registration, and session management
- name: Requests
  description: Fan commission requests
- name: Director
  description: AI Director conversation and scene card management
- name: Creator
  description: Creator dashboard and request approval workflows
- name: Scenes
  description: Scene card templates and configurations
- name: Users
  description: User profiles and settings

paths:
  /auth/register:
    post:
      tags:
        - Authentication
      summary: Register a new user
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required:
                - email
                - password
                - name

```

```

    - role
  properties:
    email:
      type: string
      format: email
    password:
      type: string
      format: password
      minLength: 8
    name:
      type: string
      minLength: 2
    role:
      type: string
      enum: [fan, creator]
    creatorHandle:
      type: string
      minLength: 3
      description: Required if role is creator
  responses:
    201:
      description: User created successfully
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/UserResponse'
    400:
      description: Invalid input or user already exists
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ErrorResponse'

```

/auth/login:

```

post:
  tags:
    - Authentication
  summary: Login user
  requestBody:
    required: true
  content:
    application/json:
      schema:
        type: object
        required:
          - email
          - password
        properties:
          email:
            type: string
            format: email
          password:
            type: string

```

```

        format: password
responses:
  200:
    description: Login successful
    content:
      application/json:
        schema:
          type: object
          properties:
            token:
              type: string
              description: JWT access token
            user:
              $ref: '#/components/schemas/UserResponse'
  401:
    description: Invalid credentials
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/ErrorResponse'

/auth/logout:
  post:
    tags:
      - Authentication
    summary: Logout user
    security:
      - BearerAuth: []
    responses:
      204:
        description: Logout successful

/auth/me:
  get:
    tags:
      - Authentication
    summary: Get current user
    security:
      - BearerAuth: []
    responses:
      200:
        description: Current user
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/UserResponse'
      401:
        description: Unauthorized

/requests:
  post:
    tags:
      - Requests

```

```

summary: Create new commission request
security:
  - BearerAuth: []
requestBody:
  required: true
  content:
    application/json:
      schema:
        type: object
        required:
          - title
          - budget
          - requestedDeliveryDate
        properties:
          title:
            type: string
            minLength: 3
          description:
            type: string
          budget:
            type: number
            format: float
            minimum: 0
          requestedDeliveryDate:
            type: string
            format: date
responses:
  201:
    description: Request created
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/RequestResponse'
  400:
    description: Invalid input
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/ErrorResponse'
  401:
    description: Unauthorized

get:
  tags:
    - Requests
  summary: List user's commission requests
  security:
    - BearerAuth: []
  parameters:
    - name: status
      in: query
      schema:
        type: string

```

```

      enum: [draft, submitted, approved, in_production, delivered, declined]
    - name: sort
      in: query
      schema:
        type: string
        enum: [created_at, updated_at, budget, delivery_date]
    - name: order
      in: query
      schema:
        type: string
        enum: [asc, desc]
  responses:
    200:
      description: List of requests
      content:
        application/json:
          schema:
            type: object
            properties:
              data:
                type: array
                items:
                  $ref: '#/components/schemas/RequestResponse'
              pagination:
                $ref: '#/components/schemas/Pagination'

/requests/{requestId}:
  get:
    tags:
      - Requests
    summary: Get request details
    security:
      - BearerAuth: []
    parameters:
      - name: requestId
        in: path
        required: true
        schema:
          type: string
          format: uuid
    responses:
      200:
        description: Request details
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/RequestDetailResponse'
      404:
        description: Request not found

  patch:
    tags:
      - Requests

```

summary: Update request (save idea, update budget, etc.)

security:

- BearerAuth: []

parameters:

- name: requestId

in: path

required: true

schema:

type: string

format: uuid

requestBody:

content:

application/json:

schema:

type: object

properties:

title:

type: string

description:

type: string

budget:

type: number

format: float

requestedDeliveryDate:

type: string

format: date

status:

type: string

enum: [draft, submitted]

responses:

200:

description: Request updated

content:

application/json:

schema:

\$ref: '#/components/schemas/RequestResponse'

/director/message:

post:

tags:

- Director

summary: Send message to AI Director

security:

- BearerAuth: []

requestBody:

required: true

content:

application/json:

schema:

type: object

required:

- requestId

- message

```

      properties:
        requestId:
          type: string
          format: uuid
        message:
          type: string
          minLength: 1
          maxLength: 2000
    responses:
      200:
        description: Director response
        content:
          application/json:
            schema:
              type: object
              properties:
                message:
                  type: string
                  description: Director's response message
                sceneCard:
                  $ref: '#/components/schemas/SceneCard'
                suggestions:
                  type: array
                  items:
                    type: string
                  description: Optional choice cards for user
      400:
        description: Invalid input

/director/choice:
  post:
    tags:
      - Director
    summary: Select a choice card from Director
    security:
      - BearerAuth: []
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required:
              - requestId
              - choiceIndex
            properties:
              requestId:
                type: string
                format: uuid
              choiceIndex:
                type: integer
                minimum: 0
    responses:

```

```

    200:
      description: Scene card updated
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/SceneCard'

/scene/{requestId}:
  get:
    tags:
      - Scenes
    summary: Get live scene card for request
    security:
      - BearerAuth: []
    parameters:
      - name: requestId
        in: path
        required: true
        schema:
          type: string
          format: uuid
    responses:
      200:
        description: Scene card
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/SceneCard'

/creator/requests:
  get:
    tags:
      - Creator
    summary: List all creator's pending requests
    security:
      - BearerAuth: []
    parameters:
      - name: status
        in: query
        schema:
          type: string
          enum: [submitted, changes_requested, approved, declined]
      - name: sort
        in: query
        schema:
          type: string
          enum: [created_at, budget, delivery_date]
      - name: page
        in: query
        schema:
          type: integer
          minimum: 1
    responses:

```

```

    200:
      description: List of creator's requests
      content:
        application/json:
          schema:
            type: object
            properties:
              data:
                type: array
                items:
                  $ref: '#/components/schemas/CreatorRequestCard'
              pagination:
                $ref: '#/components/schemas/Pagination'

/creator/requests/{requestId}:
  get:
    tags:
      - Creator
    summary: Get full request details for creator
    security:
      - BearerAuth: []
    parameters:
      - name: requestId
        in: path
        required: true
        schema:
          type: string
          format: uuid
    responses:
      200:
        description: Full request details
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/CreatorRequestDetail'

/creator/requests/{requestId}/approve:
  post:
    tags:
      - Creator
    summary: Approve a commission request
    security:
      - BearerAuth: []
    parameters:
      - name: requestId
        in: path
        required: true
        schema:
          type: string
          format: uuid
    requestBody:
      content:
        application/json:

```

```

    schema:
      type: object
      properties:
        notes:
          type: string
          description: Optional notes for fan
  responses:
    200:
      description: Request approved
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/RequestResponse'
    404:
      description: Request not found

/creator/requests/{requestId}/propose:
  post:
    tags:
      - Creator
    summary: Propose changes to a request
    security:
      - BearerAuth: []
    parameters:
      - name: requestId
        in: path
        required: true
        schema:
          type: string
          format: uuid
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required:
              - changes
            properties:
              changes:
                type: string
                description: What needs to change
              revisedPrice:
                type: number
                format: float
              revisedDeliveryDate:
                type: string
                format: date
    responses:
      200:
        description: Changes proposed
        content:
          application/json:

```

```

      schema:
        $ref: '#/components/schemas/RequestResponse'

/creator/requests/{requestId}/decline:
  post:
    tags:
      - Creator
    summary: Decline a commission request
    security:
      - BearerAuth: []
    parameters:
      - name: requestId
        in: path
        required: true
        schema:
          type: string
          format: uuid
    requestBody:
      content:
        application/json:
          schema:
            type: object
            properties:
              reason:
                type: string
    responses:
      200:
        description: Request declined
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/RequestResponse'

/creator/requests/{requestId}/ask:
  post:
    tags:
      - Creator
    summary: Ask fan a clarifying question
    security:
      - BearerAuth: []
    parameters:
      - name: requestId
        in: path
        required: true
        schema:
          type: string
          format: uuid
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object

```

```

    required:
      - question
    properties:
      question:
        type: string
        minLength: 5
  responses:
    200:
      description: Question sent
      content:
        application/json:
          schema:
            type: object
            properties:
              id:
                type: string
                format: uuid
              requestId:
                type: string
                format: uuid
              question:
                type: string
              createdAt:
                type: string
                format: date-time

```

components:

securitySchemes:

BearerAuth:

```

  type: http
  scheme: bearer
  bearerFormat: JWT

```

schemas:

UserResponse:

```

  type: object
  properties:
    id:
      type: string
      format: uuid
    email:
      type: string
      format: email
    name:
      type: string
    role:
      type: string
      enum: [fan, creator, admin]
    creatorHandle:
      type: string
    profile:
      type: object
      properties:

```

```

    avatar:
      type: string
      format: url
    bio:
      type: string
  createdAt:
    type: string
    format: date-time
  updatedAt:
    type: string
    format: date-time

```

RequestResponse:

```

  type: object
  properties:
    id:
      type: string
      format: uuid
    fanId:
      type: string
      format: uuid
    title:
      type: string
    description:
      type: string
    budget:
      type: number
      format: float
    requestedDeliveryDate:
      type: string
      format: date
    status:
      type: string
      enum: [draft, submitted, approved, in_production, delivered, declined, changes_requested]
    estimatedTotal:
      type: number
      format: float
    createdAt:
      type: string
      format: date-time
    updatedAt:
      type: string
      format: date-time

```

RequestDetailResponse:

```

  allOf:
    - $ref: '#/components/schemas/RequestResponse'
    - type: object
      properties:
        sceneCard:
          $ref: '#/components/schemas/SceneCard'
        conversation:
          type: array

```

```

    items:
      $ref: '#/components/schemas/ConversationMessage'
    timeline:
      $ref: '#/components/schemas/RequestTimeline'

```

SceneCard:

```

type: object
properties:
  id:
    type: string
    format: uuid
  requestId:
    type: string
    format: uuid
  title:
    type: string
  description:
    type: string
  components:
    type: array
    items:
      $ref: '#/components/schemas/SceneComponent'
  lineItems:
    type: array
    items:
      $ref: '#/components/schemas/LineItem'
  estimatedTotal:
    type: number
    format: float
  referenceImage:
    type: string
    format: url
  wardrobe:
    type: string
  creatorBoundaries:
    type: string
  createdAt:
    type: string
    format: date-time
  updatedAt:
    type: string
    format: date-time

```

SceneComponent:

```

type: object
properties:
  id:
    type: string
    format: uuid
  name:
    type: string
  value:
    type: string

```

```
icon:
  type: string
price:
  type: number
  format: float
isEditable:
  type: boolean
```

```
LineItem:
  type: object
  properties:
    id:
      type: string
      format: uuid
    label:
      type: string
    price:
      type: number
      format: float
    editable:
      type: boolean
```

```
ConversationMessage:
  type: object
  properties:
    id:
      type: string
      format: uuid
    requestId:
      type: string
      format: uuid
    sender:
      type: string
      enum: [fan, director]
    content:
      type: string
    suggestions:
      type: array
      items:
        type: string
    createdAt:
      type: string
      format: date-time
```

```
RequestTimeline:
  type: object
  properties:
    creatorReviewedAt:
      type: string
      format: date-time
    approvedAt:
      type: string
      format: date-time
```

```

    paymentReceivedAt:
      type: string
      format: date-time
    productionStartedAt:
      type: string
      format: date-time
    deliveredAt:
      type: string
      format: date-time
    estimatedDeliveryDate:
      type: string
      format: date

```

CreatorRequestCard:

```

type: object
properties:
  id:
    type: string
    format: uuid
  fanHandle:
    type: string
  fanAvatar:
    type: string
    format: url
  title:
    type: string
  description:
    type: string
  estimatedTotal:
    type: number
    format: float
  requestedDeliveryDate:
    type: string
    format: date
  status:
    type: string
    enum: [submitted, changes_requested, approved, declined]
  unrespondedQuestions:
    type: integer
  createdAt:
    type: string
    format: date-time

```

CreatorRequestDetail:

```

type: object
properties:
  id:
    type: string
    format: uuid
  fan:
    type: object
    properties:
      id:

```

```

      type: string
      format: uuid
    name:
      type: string
    handle:
      type: string
    avatar:
      type: string
      format: url
    sceneCard:
      $ref: '#/components/schemas/SceneCard'
    conversation:
      type: array
      items:
        $ref: '#/components/schemas/ConversationMessage'
    status:
      type: string
    timeline:
      $ref: '#/components/schemas/RequestTimeline'
    createdAt:
      type: string
      format: date-time

```

```

Pagination:
  type: object
  properties:
    total:
      type: integer
    page:
      type: integer
    pageSize:
      type: integer
    pages:
      type: integer

```

```

ErrorResponse:
  type: object
  properties:
    error:
      type: string
    message:
      type: string
    code:
      type: string
    timestamp:
      type: string
      format: date-time

```

API Implementation Notes

Authentication

- All endpoints require JWT Bearer token (except `/auth/register` and `/auth/login`)
- Token should be included in `Authorization: Bearer <token>` header
- Token expiry: 24 hours
- Refresh token: 30 days

Rate Limiting

- 100 requests per minute per user
- 10 requests per second per IP
- Creator endpoints (approve/decline): 50 per minute

Error Handling

- All errors follow standard format: `{ error, message, code, timestamp }`
- HTTP status codes: 200 (OK), 201 (Created), 204 (No Content), 400 (Bad Request), 401 (Unauthorized), 404 (Not Found), 409 (Conflict), 500 (Internal Server Error)

Pagination

- Default page size: 20
- Max page size: 100
- Use `page` and `pageSize` query parameters

Timestamps

- All timestamps are ISO 8601 format (UTC)
- Example: `2024-01-15T10:30:45Z`

Idempotency

- POST requests for approve/decline should be idempotent
- Use `Idempotency-Key` header to ensure safety on retries